

初回デート成立率を高める AI エージェントの開発

角田 直樹

1. 何を実現したいか
2. どのように実現しているか

何を実現したいか

マッチングアプリの課題

👉 出会えても **関係**が**続かない**

会話が**続かない** → 何を送ればいいか分からない

返信を考えるのが **面倒** → 継続できず途中でやめてしまう

「質」と「継続」を同時に改善する

👍 会話の継続率を高める **返信**を**自動**で作成

会話履歴とプロフィールをもとに
返信を生成

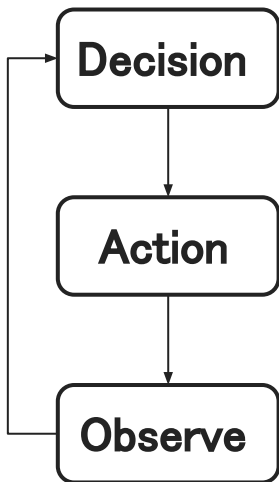
相手の温度感に
合わせた**自然**な
コミュニケーション

返信作成の手間を
削減し**継続**を支援

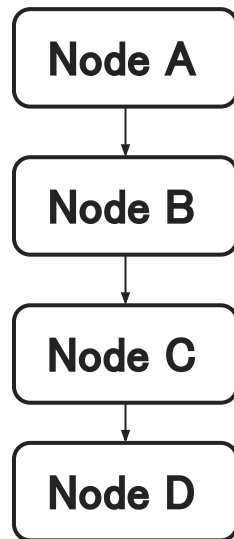
どのように実現しているか

ワークフロー設計の方針

ReAct



事前定義



両方を試して、何が最適か勉強中



ReAct

理想論最強！ 構造的に強く柔軟な構成
評価基盤が未成熟な状態では **制御が困難**

事前定義

制御性最強！ 管理しやすい構成
柔軟性が低く 未知ケースへの適応が難しい

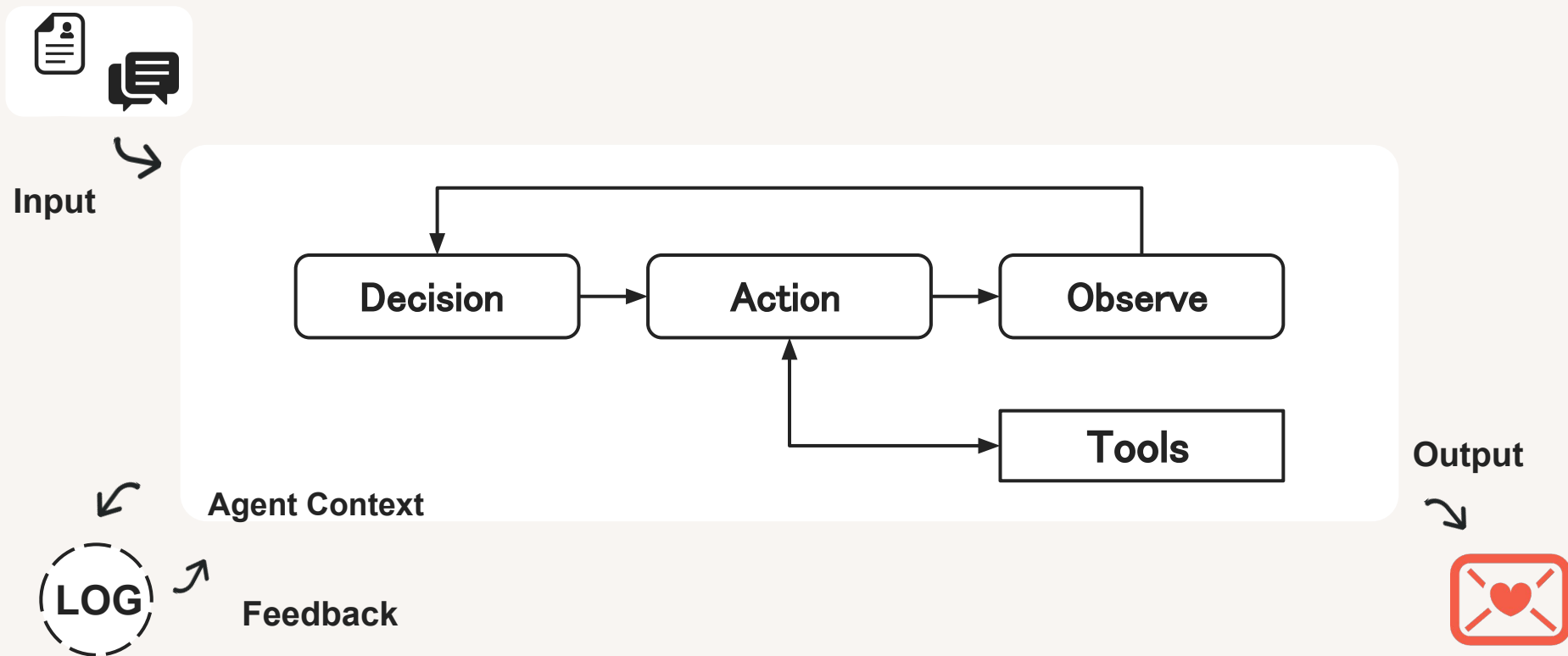
事前定義型で **評価基準を確立**
するためのログデータを取得する

※ 柔軟性よりも制御性を優先し、評価基盤を先に構築予定

※ ワークフローの構成を変更しているだけで、NodeやTool、その他のソースはどちらの構成でも使えるようにしている

※ 2026/5時点ではReActを利用中。なんだかんだループを回した方が好みの出力になったため

構成 (ReAct)



※ Agent Context: ワークフロー / プロンプト / ツール / モデル等のエージェントを構成する静的な要素の集合

この構成で実現できること

👉 最適な返信 を生成し、継続的に改善 できるエージェント

① 状況に応じた最適な意思決定

- ・会話の流れと相手の情報をもとに返信を動的に生成
- ・一貫性のある自然なコミュニケーションを実現

② 継続的に改善する仕組み

- ・ログを活用し振る舞いを評価/改善
- ・ガードレールにより品質を安定化

③ 実世界に接続した提案

- ・スケジュールを踏まえたデート提案
- ・自分のプロフィールを参照した現実的な会話

拡張性と改善性を両立する設計

① ReActベースの責務分離アーキテクチャ

事前定義型のように処理順序を固定せず、 Decision / Action / Observe のループ構成にすることで、新しい処理やツールを追加してもワークフロー全体を再設計せずに利用できる構成にしている。

② ログを前提としたノード設計

base_node を実装して全ノードのログ出力を統一することで、判断理由・実行結果を一貫した形式で記録し、後から振る舞いを検証・改善できる改善性と可観測性を確保する

③ Node / Tool 分離による安全な実行制御

ActionNode内に具体的な処理を直接持たせず、実行処理を Toolとして切り出すことで、新しい機能追加時も ActionNodeを肥大化させず、ReAct構成の柔軟性を維持したまま拡張できるようにしている。

※ ①と③は、「新しい機能・ロジック・Toolを追加しても、ワークフロー全体を再設計せず拡張できる構成にする」という共通目的を持っている。

※ ①は ReAct による「フロー」の柔軟性、③は Node / Tool 分離による「部品」の柔軟性を担保している。

👉 デート**成立率**は本人より**高い**（と書きたい）

	5月(5/23時点)	6月	7月	8月
デート回数	1			
返信回数	42			
マッチング人数	12			
デート率	8%			
アプリ利用金額	5,000円			
API利用料(概算)	1,050円(+α)			

※ 25円/返信 (ML Flowによると)

※ 返信回数 = 相手から返信が返ってきた回数

※ デート率 = デート回数 / マッチング人数